جامعة أم القرى

UMM AL-QURA UNIVERSITY

Database Project: Train System

Course Name: Introduction to Database

Instructor Name: Dr.Ghaidaa Abdulhafith

ID	NAME
445000051	Asail Hamdan
445001304	Jomanah Alghamdi
44510225	Aram Mohammad Alouni
445005754	Rema saed althqfy

Table of Contents:

1- Cover Page ……2

2- Table of Contents ……3

3- Work Distribution Table ……4

4- Section 1: Introduction Conceptual and Logical Design ……5,6

5- ERD Diagram and Relational Schema ……7

6- Section 2: Implementation code ……8-18

7- conclusion……19

Work Distribution Table:

TASK	NAME
PROJECT REPORT AND CODE COMMENTING	Asail Hamdan
ERD AND RELATIONAL SCHEMA DESIGN	Jomanah Alghamdi
WRITING PART OF THE SQL CODE	Aram Mohammad Alouni
WRITING PART OF THE SQL CODE	Rema saed althqfy

Conceptual and Logical Design

The train system manages the overall operation of trains, including trips, passengers, reservations, tickets, routes, and the employees working on each trip.

The system records its passengers, their reservations, the tickets issued for each reservation, the trips available for travel, the trains operating these trips, the routes followed by each trip, and the employees responsible for running the trips.

A Passenger is identified by their first name, last name, phone number, and a unique passenger identifier.

A Reservation is identified by its reservation time, reservation status, total fare, and a unique reservation identifier.

A Ticket is identified by its issue date and time, ticket fare, ticket status, and a unique ticket identifier.

A Trip is identified by its departure date and time, arrival date and time, trip status, and a unique trip identifier.

A Train is identified by its type, capacity, operational status, and a unique train identifier.

A Route is identified by its route name, description, main stations, total distance, number of trips, and a unique route identifier.

An Employee is identified by their first name, last name, phone number, role, salary, and a unique employee identifier.

A Passenger can make many Reservations, and each Reservation is made by one Passenger.

A Reservation can issue many Tickets, and each Ticket is issued from one Reservation.

A Passenger can book many Tickets, and each Ticket is booked by one Passenger.

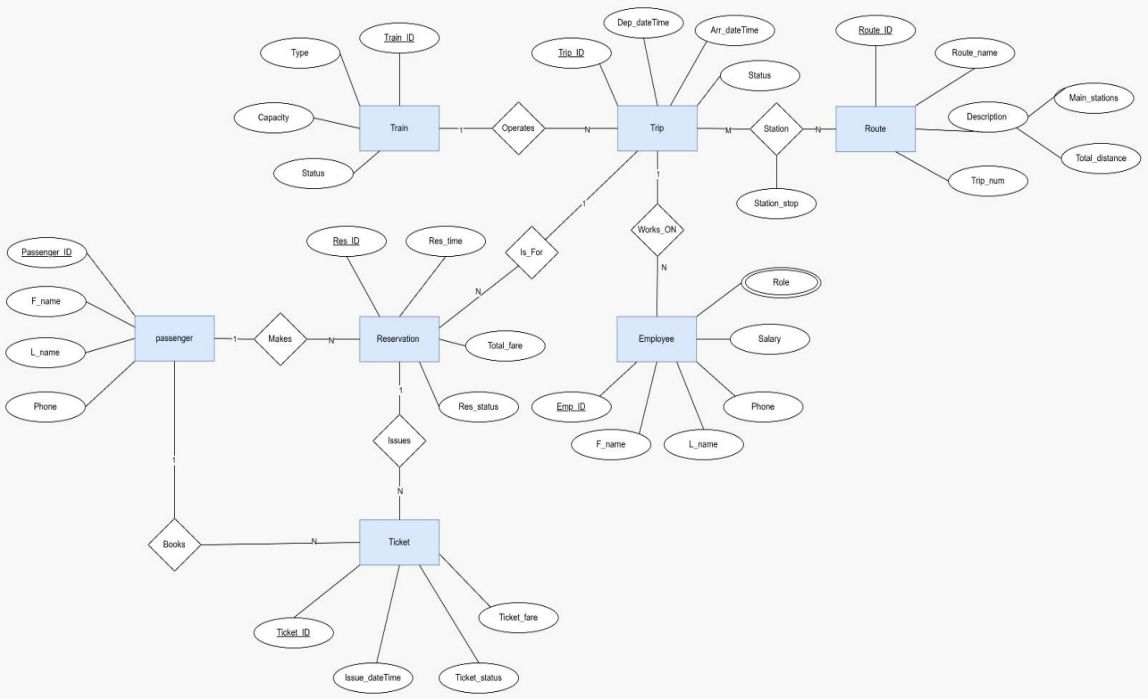
A Train can operate many Trips, and each Trip is operated by one Train.

A Trip can be for many Reservations, and each Reservation is for one Trip.

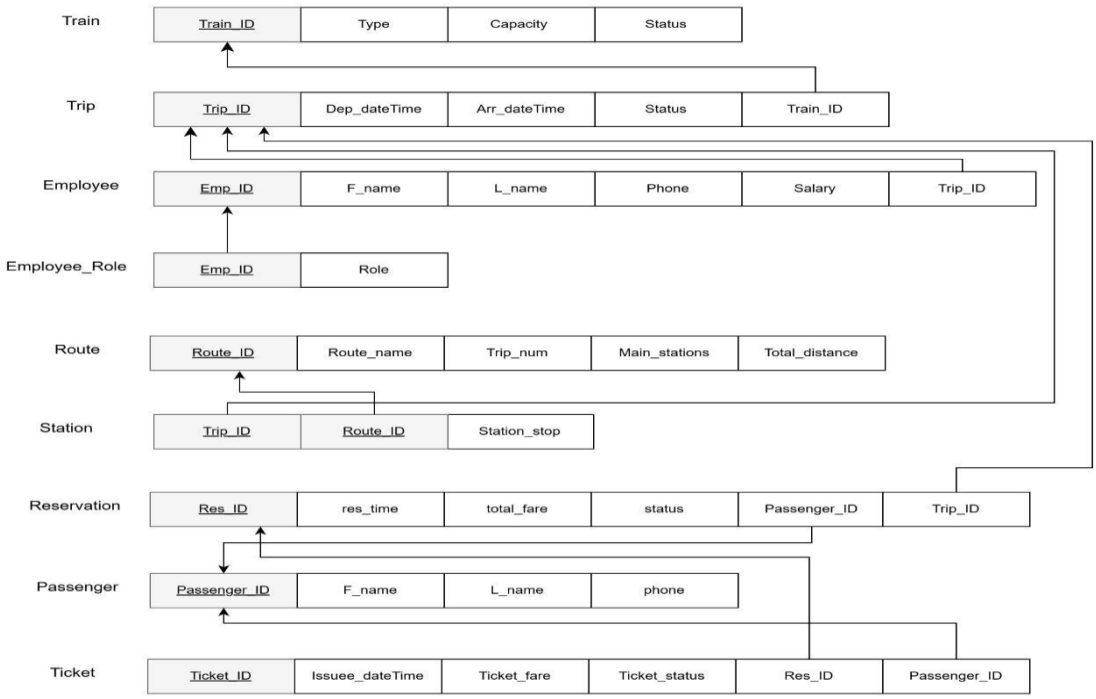
A Trip can have many Employees working on it, and each Employee works on one Trip.

A Trip can have many Station entries, and a Route can have many Station entries as well. The Station relationship includes the attribute Station stop.

ERD Diagram



Relational Schema



Implementation Description and Screenshots of all the work

```sql
CREATE DATABASE TrainSystem;
USE TrainSystem;
```

Create and use Train System database

```
75 18:44:40 CREATE DATABASE TrainSystem
76 18:44:44 USE TrainSystem
```

The output

```sql
CREATE TABLE Train (
    Train_ID INT PRIMARY KEY,
    Type VARCHAR(50) NOT NULL,
    Capacity INT CHECK (Capacity > 0),
    Status VARCHAR(30) NOT NULL
);
```

Creates the Train table

```
77 19:03:49 CREATE TABLE Train (  Train_ID INT PRIMARY KEY,  Type VARCHAR(50) ...
```

The output

```sql
INSERT INTO Train (Train_ID, Type, Capacity, Status)
VALUES
(1, 'Individual', 180, 'Active'),
(2, 'Family', 200, 'Active'),
(3, 'First Class', 150, 'Active'),
(4, 'Family', 120, 'Under Maintenance'),
(5, 'Individual', 160, 'Inactive');
```

Insert data into Train table

```
78 19:26:59 INSERT INTO Train (Train_ID, Type, Capacity, Status) VALUES
```

The output

```
20• SELECT * FROM Train;
```

Displays the Train table

Result Grid | Filter Rows: | Edit:

Train_ID	Type	Capacity	Status
1	Individual	180	Active
2	Family	200	Active
3	First Class	150	Active
4	Family	120	Under Maintenance
5	Individual	160	Inactive
NULL	NULL	NULL	NULL

The output

```
CREATE TABLE Trip (
    Trip_ID INT PRIMARY KEY,
    Dep_DateTime DATETIME NOT NULL,
    Arr_DateTime DATETIME NOT NULL,
    Status VARCHAR(30),
    Train_ID INT,
    FOREIGN KEY (Train_ID) REFERENCES Train(Train_ID)
);
```

Creates the Trip table and Trip-Train_ID is a foreign key that references Train-Train_ID.

Relationship: One Train can have many Trips One to Many.

80 19:59:57 CREATE TABLE Trip (Trip_ID INT PRIMARY KEY,

The output

```
INSERT INTO Trip (Trip_ID, Dep_DateTime, Arr_DateTime, Status, Train_ID)
VALUES
(1, '2025-11-10 08:00:00', '2025-11-10 12:00:00', 'Completed', 1),
(2, '2025-11-11 09:00:00', '2025-11-11 13:00:00', 'Scheduled', 3),
(3, '2025-11-12 10:30:00', '2025-11-12 14:30:00', 'Cancelled', 2),
(4, '2025-11-13 06:00:00', '2025-11-13 09:30:00', 'Completed', 4),
(5, '2025-11-14 15:00:00', '2025-11-14 19:30:00', 'Scheduled', 5);
```

Insert data into Trip table

81 20:15:57 INSERT INTO Trip (Trip_ID,

The output

```
41 • SELECT * FROM Trip;
```

Displays the Trip table

Result Grid | Filter Rows: | Edit: | Export/Impor

Trip_ID	Dep_DateTime	Arr_DateTime	Status	Train_ID
1	2025-11-10 08:00:00	2025-11-10 12:00:00	Completed	1
2	2025-11-11 09:00:00	2025-11-11 13:00:00	Scheduled	3
3	2025-11-12 10:30:00	2025-11-12 14:30:00	Cancelled	2
4	2025-11-13 06:00:00	2025-11-13 09:30:00	Completed	4
5	2025-11-14 15:00:00	2025-11-14 19:30:00	Scheduled	5

The output

```sql
CREATE TABLE Route (
    Route_ID INT PRIMARY KEY,
    Route_name VARCHAR(100) NOT NULL,
    Trip_num INT,
    Main_stations VARCHAR(255) NOT NULL,
    Total_distance DECIMAL(6,2) CHECK (Total_distance > 0)
);
```

Creates the Route table

```
83  20:27:16  CREATE TABLE Route (
```

The output

```sql
INSERT INTO Route (Route_ID, Route_name, Trip_num, Main_stations, Total_distance)
VALUES
(1, 'Makkah-Jeddah', 101, 'Al Rusaifah, Al Sulaymaniyah', 85.0),
(2, 'Jeddah-Madinah', 102, 'Al Sulaymaniyah, King Abdullah Economic City, Madinah', 420.0),
(3, 'Makkah-Taif', 103, 'Al Rusaifah, Al Hada, Taif', 120.0),
(4, 'Riyadh-Dammam', 104, 'Riyadh, Hofuf, Dammam', 400.0),
(5, 'Jeddah-Yanbu', 105, 'Al Sulaymaniyah, Rabigh, Yanbu', 350.0);
```

Insert data into Route table

```
84  20:36:30  INSERT INTO Route (Route_ID
```

The output

```sql
61  SELECT * FROM Route;
```

Displays the Route table

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

Route_ID	Route_name	Trip_num	Main_stations	Total_distance
1	Makkah–Jeddah	101	Al Rusaifah, Al Sulaymaniyah	85.00
2	Jeddah–Madinah	102	Al Sulaymaniyah, King Abdullah Economic City, ...	420.00
3	Makkah–Taif	103	Al Rusaifah, Al Hada, Taif	120.00
4	Riyadh–Dammam	104	Riyadh, Hofuf, Dammam	400.00
5	Jeddah–Yanbu	105	Al Sulaymaniyah, Rabigh, Yanbu	350.00

The output

```sql
CREATE TABLE Station (
    Trip_ID INT,
    Route_ID INT,
    Station_stop VARCHAR(100),
    FOREIGN KEY (Trip_ID) REFERENCES Trip(Trip_ID),
    FOREIGN KEY (Route_ID) REFERENCES Route(Route_ID)
);
```

Creates the Station associative table to represent the relationship between Trip and Route. Station.Trip_ID is a foreign key that references Trip (Trip_ID). Station.Route_ID is a foreign key that references Route (Route_ID). This design resolves the many to many relationships between Trip and Route and stores the attribute Station_Stop for each station entry.

```
86  20:45:16  CREATE TABLE Station (   Trip_ID
```

The output

```sql
INSERT INTO Station (Trip_ID, Route_ID, Station_stop)
VALUES
    (1, 2, 'Madinah'),
    (2, 1, 'Al Sulaymaniyah'),
    (3, 5, 'Yanbu'),
    (4, 4, 'Dammam'),
    (5, 3, 'Taif');
```

Insert data into stations table

```
87  20:46:00  INSERT INTO Station (Trip_ID,
```

The output

```sql
SELECT * FROM Station;
```

Displays the Station table

Result Grid | Filter Rows:

Trip_ID	Route_ID	Station_stop
1	2	Madinah
2	1	Al Sulaymaniyah
3	5	Yanbu
4	4	Dammam
5	3	Taif

The output

```sql
CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    F_name VARCHAR(50) NOT NULL,
    L_name VARCHAR(50) NOT NULL,
    Phone VARCHAR(15) UNIQUE,
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    Trip_ID INT,
    FOREIGN KEY (Trip_ID) REFERENCES Trip(Trip_ID)
    );
```

Creates the Employee table and Employee-Trip_ID is a foreign key that references Trip-Trip_ID.

Relationship: One Trip can have many Employees One-to-Many.

```
89  20:59:55  CREATE TABLE Employee (
```

The output

```sql
INSERT INTO Employee (Emp_ID, F_name, L_name, Phone, Salary, Trip_ID)
VALUES
(1, 'Reema', 'Saed', '0557324189', 7200.00, 1),
(2, 'Jumanah', 'Al-Ghamdi', '0562895403', 6800.00, 1),
(3, 'Aram', 'Mohammed', '0573941256', 6500.00, 2),
(4, 'Asayel', 'Hamdan', '0581027394', 7000.00, 3),
(5, 'Fahad', 'Al-Qahtani', '0598742635', 7500.00, 5);
```

Insert data into Employee table

```
90  21:01:00  INSERT INTO Employee (
```

The output

```
SELECT * FROM Employee;
```

Result Grid | Filter Rows: | Edit:

Emp_ID	F_name	L_name	Phone	Salary	Trip_ID
1	Reema	Saed	0557324189	7200.00	1
2	Jumanah	Al-Ghamdi	0562895403	6800.00	1
3	Aram	Mohammed	0573941256	6500.00	2
4	Asayel	Hamdan	0581027394	7000.00	3
5	Fahad	Al-Qahtani	0598742635	7500.00	5
NULL	NULL	NULL	NULL	NULL	NULL

Displays the Employee table

The output

```
CREATE TABLE Employee_Role (
    Emp_ID INT,
    Role VARCHAR(50) NOT NULL,
    FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID)
);
```

Creates the Employee_Role table to store the multi-valued attribute Role for each Employee.
Employee_Role.Emp_ID is a foreign key referencing Employee (Emp_ID).
One Employee can have many Role records.

92 21:10:05 CREATE TABLE Employee_Role (

The output

```
INSERT INTO Employee_Role (Emp_ID, Role)
VALUES
    (1, 'Supervisor'),
    (2, 'Ticket Officer'),
    (3, 'Train Driver'),
    (4, 'Station Manager'),
    (5, 'Technician');
```

Insert data into Employee_Role table

93 21:12:20 INSERT INTO Employee_Role (Er

The output

```
SELECT * FROM Employee_Role;
```

Result Grid | Filter Rows:

Emp_ID	Role
1	Supervisor
2	Ticket Officer
3	Train Driver
4	Station Manager
5	Technician

Displays the Employee Role table

The output

```sql
CREATE TABLE Passenger (
    Passenger_ID INT PRIMARY KEY,
    F_name VARCHAR(50) NOT NULL,
    L_name VARCHAR(50) NOT NULL,
    Phone VARCHAR(15) UNIQUE
);
```

Creates the Passenger table

```
97  21:22:08  CREATE TABLE Passenger
```

The output

```sql
INSERT INTO Passenger (Passenger_ID, F_name, L_name, Phone)
VALUES
    (1, 'Yara', 'Al-Fahad', '0557324189'),
    (2, 'Lina', 'Al-Mutairi', '0562895403'),
    (3, 'Sara', 'Al-Harbi', '0573941256'),
    (4, 'Amal', 'Al-Qahtani', '0581027394'),
    (5, 'Nora', 'Al-Ghamdi', '0598742635');
```

Insert data into Passenger table

```
98  21:24:10  INSERT INTO Passenger
```

The output

```sql
142  SELECT * FROM Passenger;
```

Result Grid | Filter Rows: | Edit:

Passenger_ID	F_name	L_name	Phone
1	Yara	Al-Fahad	0557324189
2	Lina	Al-Mutairi	0562895403
3	Sara	Al-Harbi	0573941256
4	Amal	Al-Qahtani	0581027394
5	Nora	Al-Ghamdi	0598742635
NULL	NULL	NULL	NULL

Displays the Passenger table

The output

```sql
CREATE TABLE Reservation (
    Res_ID INT PRIMARY KEY,
    Res_time DATETIME NOT NULL,
    Total_fare DECIMAL(10,2),
    Status VARCHAR(30),
    Passenger_ID INT,
    Trip_ID INT,
    FOREIGN KEY (Passenger_ID) REFERENCES Passenger(Passenger_ID),
    FOREIGN KEY (Trip_ID) REFERENCES Trip(Trip_ID)
);
```

Creates the Reservation table and Reservation-Passenger_ID is a foreign key that references Passenger-Passenger_ID.
Relationship: One Passenger can have many Reservations One to Many.

Reservation-Trip_ID is a foreign key that references Trip-Trip_ID.
Relationship: One Trip can have many Reservations One to Many.

```
100  21:26:41  CREATE TABLE Reservation (
```

The output

```sql
INSERT INTO Reservation (Res_ID, Res_time, Total_fare, Status, Passenger_ID, Trip_ID)
VALUES
    (1, '2025-11-10 08:30:00', 150.00, 'Confirmed', 1, 1),
    (2, '2025-11-11 09:45:00', 200.00, 'Confirmed', 2, 2),
    (3, '2025-11-12 14:20:00', 180.00, 'Pending', 3, 3),
    (4, '2025-11-13 16:00:00', 220.00, 'Cancelled', 4, 4),
    (5, '2025-11-14 12:15:00', 170.00, 'Confirmed', 5, 5);
```

Insert data into Reservation table

101 21:29:13 INSERT INTO Reservation (

The output

```sql
SELECT * FROM Reservation;
```

Displays the Reservation table

Result Grid | Filter Rows: | Edit: | Export/Impor

Res_ID	Res_time	Total_fare	Status	Passenger_ID	Trip_ID
1	2025-11-10 08:30:00	150.00	Confirmed	1	1
2	2025-11-11 09:45:00	200.00	Confirmed	2	2
3	2025-11-12 14:20:00	180.00	Pending	3	3
4	2025-11-13 16:00:00	220.00	Cancelled	4	4
5	2025-11-14 12:15:00	170.00	Confirmed	5	5
NULL	NULL	NULL	NULL	NULL	NULL

The output

```sql
CREATE TABLE Ticket (
    Ticket_ID INT PRIMARY KEY,
    Issuee_dateTime DATETIME NOT NULL,
    Ticket_fare DECIMAL(10,2),
    Ticket_status VARCHAR(30),
    Res_ID INT,
    Passenger_ID INT,
    FOREIGN KEY (Res_ID) REFERENCES Reservation(Res_ID),
    FOREIGN KEY (Passenger_ID) REFERENCES Passenger(Passenger_ID)
);
```

Creates the Reservation table and Ticket-Res_ID is a foreign key that references Reservation.Res_ID.
Relationship: One Reservation can have many Tickets One to Many.

Ticket-Passenger_ID is a foreign key that references Passenger-Passenger_ID.
Relationship: One Passenger can have many Tickets One to Many.

103 21:32:34 CREATE TABLE Ticket

The output

```sql
INSERT INTO Ticket (Ticket_ID, Issuee_dateTime, Ticket_fare, Ticket_status, Res_ID, Passenger_ID)
VALUES
    (1, '2025-11-08 08:00:00', 150.00, 'Active', 1, 1),
    (2, '2025-11-09 09:30:00', 200.00, 'Used', 2, 2),
    (3, '2025-11-10 10:15:00', 180.00, 'Cancelled', 3, 3),
    (4, '2025-11-11 11:45:00', 220.00, 'Active', 4, 4),
    (5, '2025-11-12 12:30:00', 170.00, 'Used', 5, 5);
```

Insert data into Ticket table

104 21:34:23 INSERT INTO Ticket

The output

```sql
SELECT * FROM Ticket;
```

Displays the Ticket table

Result Grid | Filter Rows: | Edit: | Export/Import:

Ticket_ID	Issuee_dateTime	Ticket_fare	Ticket_status	Res_ID	Passenger_ID
1	2025-11-08 08:00:00	150.00	Active	1	1
2	2025-11-09 09:30:00	200.00	Used	2	2
3	2025-11-10 10:15:00	180.00	Cancelled	3	3
4	2025-11-11 11:45:00	220.00	Active	4	4
5	2025-11-12 12:30:00	170.00	Used	5	5
NULL	NULL	NULL	NULL	NULL	NULL

The output

Active trains with capacity > 150

```sql
SELECT Train_ID,
Type,
Capacity,
Status
FROM Train
WHERE
Status = 'Active' AND Capacity > 150;
```

Result Grid | Filter Rows: | Edit:

Train_ID	Type	Capacity	Status
1	Individual	180	Active
2	Family	200	Active
NULL	NULL	NULL	NULL

Routes with distance > 300 km

```sql
SELECT
    Route_ID,
    Route_name,
    Trip_num,
    Main_stations,
    Total_distance
FROM
    Route
WHERE
    Total_distance > 300.00;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

Route_ID	Route_name	Trip_num	Main_stations	Total_distance
2	Jeddah–Madinah	102	Al Sulaymaniyah, King Abdullah Economic City, ...	420.00
4	Riyadh–Dammam	104	Riyadh, Hofuf, Dammam	400.00
5	Jeddah–Yanbu	105	Al Sulaymaniyah, Rabigh, Yanbu	350.00
NULL	NULL	NULL	NULL	NULL

Calculate average salary for employees on Trip ID 1

```sql
SELECT
    AVG(Salary) AS avg_salary_Trip1
FROM
    Employee
WHERE
    Trip_ID = 1;
```

Result Grid | Filter Rows: | Export:

avg_salary_Trip1
7000.000000

Count passengers by family name

```sql
SELECT
    L_name, COUNT(Passenger_ID) AS Passengers_by_Family
FROM
    Passenger
GROUP BY
    L_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

L_name	Passengers_by_Family
Al-Fahad	1
Al-Mutairi	1
Al-Harbi	1
Al-Qahtani	1
Al-Ghamdi	1

employee details, role, and trip schedule

```
SELECT
    E.F_name AS Employee_Name,
    E.L_name AS Employee_Family,
    E.Salary,
    ER.Role,
    T.Dep_DateTime,
    T.Arr_DateTime
FROM
    Employee E
INNER JOIN
    Employee_Role ER ON E.Emp_ID = ER.Emp_ID
INNER JOIN
    Trip T ON E.Trip_ID = T.Trip_ID;
```

Employee_Name	Employee_Family	Salary	Role	Dep_DateTime	Arr_DateTime
Reema	Saed	7200.00	Supervisor	2025-11-10 08:00:00	2025-11-10 12:00:00
Jumanah	Al-Ghamdi	6800.00	Ticket Officer	2025-11-10 08:00:00	2025-11-10 12:00:00
Aram	Mohammed	6500.00	Train Driver	2025-11-11 09:00:00	2025-11-11 13:00:00
Asayel	Hamdan	7000.00	Station Manager	2025-11-12 10:30:00	2025-11-12 14:30:00
Fahad	Al-Qahtani	7500.00	Technician	2025-11-14 15:00:00	2025-11-14 19:30:00

Select employees by Trip ID 1 using a subquery

```sql
SELECT
    F_name,
    L_name,
    Trip_ID
FROM Employee
WHERE
    Trip_ID IN (
        SELECT Trip_ID
        FROM Trip
        WHERE Trip_ID = 1
    );
```

F_name	L_name	Trip_ID
Reema	Saed	1
Jumanah	Al-Ghamdi	1

Create a reusable view for the Crew Report

```sql
CREATE VIEW Trip_Crew_Details AS
SELECT
    E.Emp_ID,
    E.F_name AS Employee_FirstName,
    E.L_name AS Employee_LastName,
    ER.Role,
    T.Trip_ID,
    T.Dep_DateTime
FROM
    Employee E
JOIN
    Employee_Role ER ON E.Emp_ID = ER.Emp_ID
JOIN
    Trip T ON E.Trip_ID = T.Trip_ID;
SELECT * FROM Trip_Crew_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Emp_ID	Employee_FirstName	Employee_LastName	Role	Trip_ID	Dep_DateTime
1	Reema	Saed	Supervisor	1	2025-11-10 08:00:00
2	Jumanah	Al-Ghamdi	Ticket Officer	1	2025-11-10 08:00:00
3	Aram	Mohammed	Train Driver	2	2025-11-11 09:00:00
4	Asayel	Hamdan	Station Manager	3	2025-11-12 10:30:00
5	Fahad	Al-Qahtani	Technician	5	2025-11-14 15:00:00

In conclusion of this report, we have presented all stages of the "Train System Database" project, which represents a practical application of the concepts acquired in the Database Design and Management course.

The project encompassed designing the conceptual and logical structure of the system via the Entity-Relationship Diagram (ERD) and the Relational Schema. This was followed by the transition to the actual implementation phase using SQL. These efforts successfully resulted in the construction of an effective database, which was tested through multiple queries that proved its efficiency in managing the core data for trips, trains, employees, and passengers, thereby confirming the achievement of the project's academic and functional objectives.